



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 5

“Contador activado por flancos”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 13 de febrero de 2025

INTRODUCCIÓN

Contador activado por flancos

Los contadores activados por flancos (Edge-Triggered Counters) son la base de la electrónica digital síncrona moderna. Su característica principal es que no cambian de estado simplemente porque haya voltaje en la señal de reloj, sino en el instante exacto en que la señal cambia de nivel.

¿Qué es un flanco?

En una señal digital cuadrada, el paso de un nivel lógico a otro no es instantáneo, pero ocurre en un periodo de tiempo extremadamente corto (nanosegundos). Existen dos tipos de transiciones que pueden disparar un contador:

- **Flanco de Subida (Positive Edge / Rising Edge):** Cuando la señal pasa de un bajo lógico (0) a un alto lógico (1). Se representa con una flecha hacia arriba.
- **Flanco de Bajada (Negative Edge / Falling Edge):** Cuando la señal pasa de un alto lógico (1) a un bajo lógico (0). Se representa con una flecha hacia abajo

Flip-Flop como unidad básica

Para que un contador sea activado por flanco, debe estar construido con Flip-Flops disparados por flanco (tipo D o tipo JK).

A diferencia de un "Latch" (que es activado por nivel y deja pasar los datos mientras la señal esté en alto), el Flip-Flop activado por flanco tiene un circuito detector de transiciones interno. Este circuito genera un pulso de muestreo interno tan breve que el estado solo puede cambiar en ese momento preciso, ignorando cualquier cambio en las entradas antes o después del flanco.

Importancia en el sincronismo

- **Inmunidad al Ruido:** Si la señal de reloj tiene un poco de ruido mientras está en nivel alto, un contador por nivel podría cambiar erróneamente. Un contador por flanco es mucho más difícil de engañar.
- **Evitar Carreras Críticas (Race Conditions):** En un contador síncrono, queremos que todos los bits cambien exactamente al mismo tiempo. El flanco proporciona ese "instante único" de disparo para todo el sistema.
- **Temporización Precisa:** Permite que el sistema sea predecible. Sabes exactamente que el conteo aumentará en la transición, lo que facilita el cálculo de frecuencias y tiempos de ejecución.

Representación en esquemáticos

En los diagramas de circuitos, se puede identificar si un contador es activado por flanco buscando un pequeño triángulo en la entrada de reloj (Clock):

- **Triángulo solo:** Activado por flanco de subida.
- **Triángulo con un círculo afuera:** Activado por flanco de bajada.

Aplicación

Cuando se configuran los Timers en CodeVisionAVR, a menudo nos encontramos con opciones de configuración para los pines de reloj externo (T0 o T1). El microcontrolador permite elegir:

- Contar en cada flanco de subida de un sensor.
- Contar en cada flanco de bajada.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 5, el circuito a armar es el siguiente:

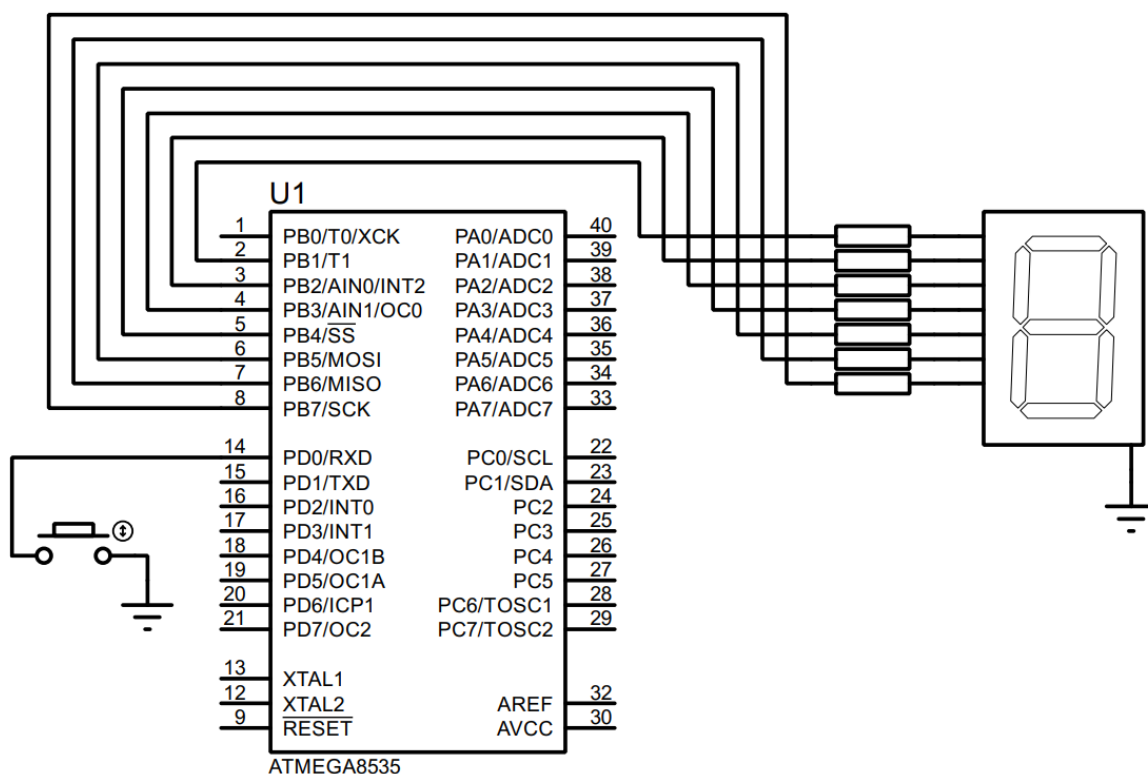


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, el registro B se configura como de salida (PB0 – PB6) mientras que el registro D como de entrada (PD0), activando la resistencia de pull-up.

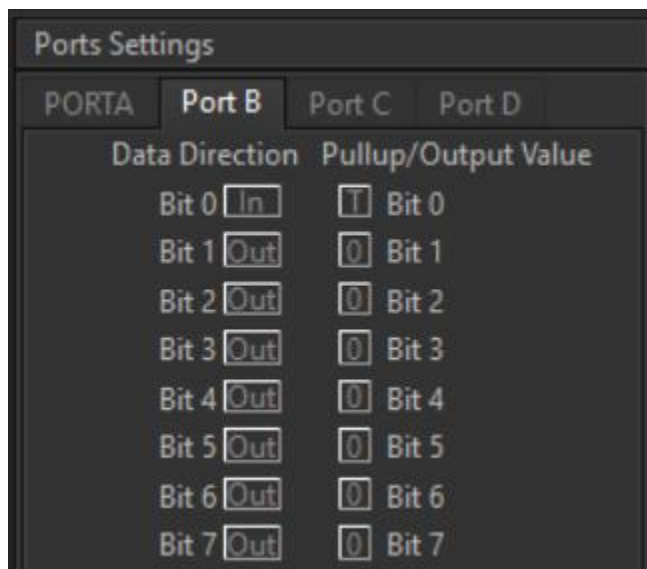


Imagen 2. Configuración del puerto B.

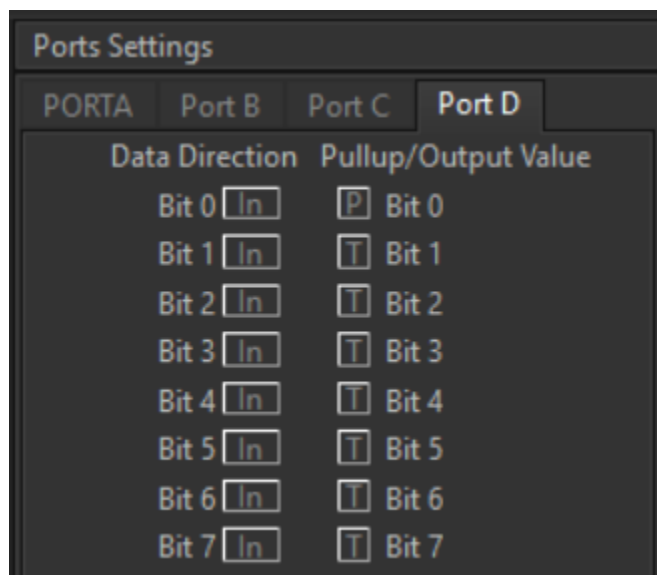


Imagen 3. Configuración del puerto D.

Para poder hacer el contador es necesario interpretar el número correspondiente del display como un número hexadecimal, para ello se realizó la siguiente tabla, tomando en cuenta lo que se indicó en la práctica 3:

Número Display	Combinaciones								Valor hexadecimal
	g	f	e	d	c	b	a		
0	0	1	1	1	1	1	1	0	0x7e
1	0	0	0	0	1	1	0	0	0x0c
2	1	0	1	1	0	1	1	0	0xb6
3	1	0	0	1	1	1	1	0	0x9e
4	1	1	0	0	1	1	0	0	0xcc

5	1	1	0	1	1	0	1	0	0xda
6	1	1	1	1	1	0	1	0	0xfa
7	0	0	0	0	1	1	1	0	0x0e
8	1	1	1	1	1	1	1	0	0xfe
9	1	1	0	1	1	1	1	0	0xde
A	1	1	1	0	1	1	1	0	0xee
B	1	1	1	1	1	0	0	0	0xf8
C	0	1	1	1	0	0	1	0	0x72
D	1	0	1	1	1	1	0	0	0xbc
E	1	1	1	1	0	0	1	0	0xf2
F	1	1	1	0	0	0	1	0	0xe2

Tabla 1. Representaci3n hexadecimal para los n3meros del 0 al 9 y del A a la F en hexadecimal.

C3digo del circuito de CodeVision

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr3ctica 5 'Contador activado por flancos'
8  Version : 1.0
9  Date    : 11/02/2026
10 Author  : Garc3a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #define BOTON PIND.0

```

```

27 // Declare your global variables here
28
29 void main(void) {
30     // Declare your Local variables here
31     bit estado_boton_anterior, estado_boton_actual;
32     unsigned char numero_bcd;
33     const char bcd[10] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0xe, 0xfe, 0xde};
34
35     // Input/Output Ports initialization
36     // Port A initialization
37     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
38     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
39     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
40     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
41
42     // Port B initialization
43     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
44     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
45     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
46     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
47
48     // Port C initialization
49     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
50     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
51     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
52     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
53
54     // Port D initialization
55     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
56     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
57     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=P
58     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (1<<PORTD0);
59
60     // Timer/Counter 0 initialization
61     // Clock source: System Clock
62     // Clock value: Timer 0 Stopped
63     // Mode: Normal top=0xFF
64     // OC0 output: Disconnected
65     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
66     TCNT0=0x00;
67     OCR0=0x00;
68
69     // Timer/Counter 1 initialization
70     // Clock source: System Clock
71     // Clock value: Timer1 Stopped
72     // Mode: Normal top=0xFFFF
73     // OC1A output: Disconnected
74     // OC1B output: Disconnected
75     // Noise Canceler: Off
76     // Input Capture on Falling Edge
77     // Timer1 Overflow Interrupt: Off
78     // Input Capture Interrupt: Off
79     // Compare A Match Interrupt: Off
80     // Compare B Match Interrupt: Off
81     TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
82     TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
83     TCNT1H=0x00;
84     TCNT1L=0x00;
85     ICR1H=0x00;
86     ICR1L=0x00;
87     OCR1AH=0x00;
88     OCR1AL=0x00;
89     OCR1BH=0x00;
90     OCR1BL=0x00;

```

```

92 // Timer/Counter 2 initialization
93 // Clock source: System Clock
94 // Clock value: Timer2 Stopped
95 // Mode: Normal top=0xFF
96 // OC2 output: Disconnected
97 ASSR=(0<<AS2);
98 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
99 TCNT2=0x00;
100 OCR2=0x00;
101
102 // Timer(s)/Counter(s) Interrupt(s) initialization
103 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
104
105 // External Interrupt(s) initialization
106 // INT0: Off
107 // INT1: Off
108 // INT2: Off
109 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
110 MCUCSR=(0<<ISC2);
111
112 // USART initialization
113 // USART disabled
114 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);
115
116 // Analog Comparator initialization
117 // Analog Comparator: Off
118 // The Analog Comparator's positive input is
119 // connected to the AIN0 pin
120 // The Analog Comparator's negative input is
121 // connected to the AIN1 pin
122 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
123 SFIOR=(0<<ACME);
124
125 // ADC initialization
126 // ADC disabled
127 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
128
129 // SPI initialization
130 // SPI disabled
131 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
132
133 // TWI initialization
134 // TWI disabled
135 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```



```
137 while (1) {  
138     // Place your code here  
139     // Determinar el estado del 'estado_boton_actual'  
140     if (!BOTON) {  
141         estado_boton_actual = 0;  
142     } else {  
143         estado_boton_actual = 1;  
144     }  
145  
146     // Detectar el cambio de flanco de 1 a 0  
147     if (!estado_boton_actual && estado_boton_anterior) {  
148         numero_bcd++;  
149     }  
150  
151     // Reiniciar el conteo al llegar a 10  
152     if (numero_bcd == 10) {  
153         numero_bcd = 0;  
154     }  
155  
156     // Actualizamos valores  
157     PORTB = bcd[numero_bcd];  
158     estado_boton_anterior = estado_boton_actual;  
159 }  
160 }
```

Circuito físico

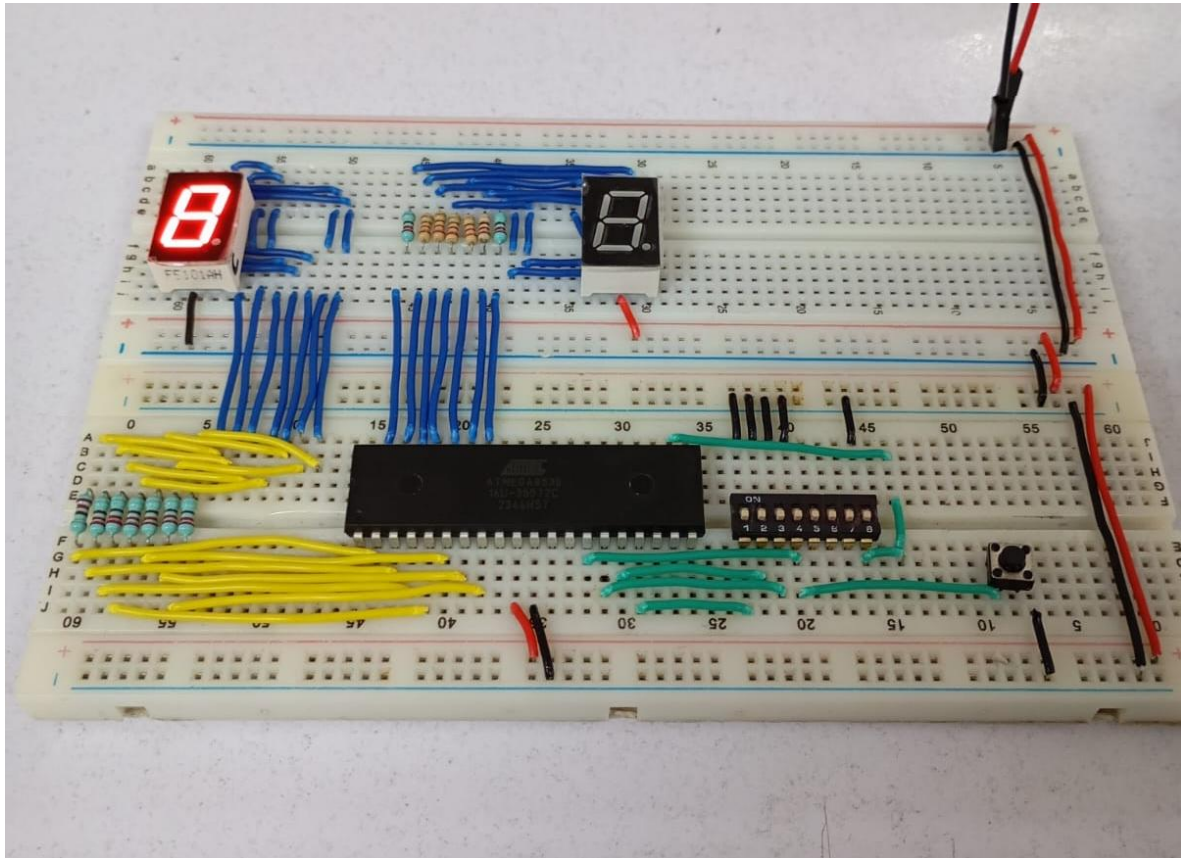


Imagen 3. Circuito físico de la práctica 5.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Para solucionar parcialmente el problema de los rebotes en esta práctica se implementa la detección por flancos, en este caso el flanco de subida el cual en nuestro caso es de 1 a 0, debido a que la resistencia de pull-up se encuentra activada. Por ello en la entrada por defecto se lee un uno lógico y cuando el botón es presionado este cambia a cero.

En el código se lee el estado actual del botón (es decir si este se presiona o no), y se compara junto con el estado anterior del botón, en este caso si el estado actual indica que el botón fue presionado y que el estado anterior indica que el botón no lo estuvo (`!estado_boton_actual & estado_boton_anterior`), entonces el contador se incrementa.

En resumen, se detecta si el botón fue presionado y soltado. El hecho de tomar en cuenta el estado anterior del botón permite despreciar la detección de los rebotes, sin embargo, a pesar de esto el problema no se soluciona del todo, pues aún estos se siguen presentado aunque en menor medida.

En la siguiente práctica este problema se solucionará por completo,

- **Meléndez Macedonio Rodrigo**

Para el desarrollo de esta práctica se trabajó sobre el mismo circuito de la anterior práctica, pues en este caso se intentó dar solución al principal problema de rebote que tenía el contador 0 a 9 mediante el uso de flancos de subida y bajada, que en este caso utilizamos el flanco de subida. Por lo tanto, se realizaron cambios en el código e integramos dos variables para identificar el estado del botón en el que se encuentra (0 o 1). Utilizamos condicionales “if” para saber si el botón fue presionado y si el botón no fue presionado anteriormente, de esta forma, el contador puede incrementar sin problema cada vez que el botón es presionado. Sin embargo, a pesar de que esta implementación es mejor que la anterior, aun no elimina el problema por completo, pues hay veces en las que el botón es presionado y el valor del contador aumenta más de una vez. Para solucionar este problema, se necesita una última implementación que se verá en la práctica posterior a esta.

BIBLIOGRAFÍA

(s.f.). *Clock Signals and Triggers*. Electronic Tutorials. Recuperado el 10 de febrero de 2026.
https://www.electronics-tutorials.ws/sequential/seq_2.html